

Ali Alemami

Amman, Jordan | ali.alemami01@gmail.com | +962 780 020 203 | github.com/ali-alemami | ali-alemami.github.io | linkedin.com/in/ali-alemami

CS student focused on backend & systems engineering — low-level systems programming (POSIX APIs, AST parsing, multithreading, concurrency, memory models) in **C/C++**, plus hands-on **C#/ .NET** application architecture and **SQL Server** database engineering. Seeking an entry-level backend, systems, or software engineering role.

EDUCATION

The Hashemite University — BSc in Computer Science 2022 – present
42 Amman — Common Core, Level 4.7/21, 21/24 core projects complete (peer-reviewed, project-based) 2025 – present

TECHNICAL SKILLS

Languages: C (POSIX/C99), C++ (98/11/17), C# (.NET), SQL (T-SQL), Bash
Systems & Low-Level: Unix process control, AST parsing, multithreading (pthreads), mutex synchronization, sockets, UNIX IPC, signals
CS Fundamentals: OOP & Design Patterns, Data Structures & Algorithms, Memory Management (Valgrind/ASan), Computer Networks
Tools & Infrastructure: Git, Linux, Docker, Docker Compose, NGINX, GDB, Valgrind, SQL Server, ADO.NET, Makefiles

42 CORE CURRICULUM — 21/24 PROJECTS COMPLETE

- **Multithreading & Concurrency (C — philo):** Simulated Dijkstra's Dining Philosophers problem using POSIX threads (pthreads) and mutex locks; implemented microsecond-precision state monitoring to guarantee zero data races, starvation prevention, and deadlock-free synchronization.
- **DevOps & Container Infrastructure (inception):** Orchestrated a secure multi-service infrastructure using Docker Compose; configured isolated containers for NGINX with TLS/SSL encryption, WordPress (PHP-FPM), and MariaDB with dedicated storage volumes, non-root daemons, and internal networking.
- **Unix Shell & AST Engine (C — minishell):** Engineered a POSIX-compliant shell with an Abstract Syntax Tree (AST) parser to evaluate operator precedence ([, &&, ||), nested subshell forks, heredocs, variable expansions, and signal-safe process control; audited for zero fd leaks via Valgrind.
- **3D Graphics & Raytracing (C — mini_rt):** Built a 3D raytracing engine from scratch in C; implemented vector linear algebra, ray-primitive geometric intersections (spheres, planes, cylinders), and Phong illumination physics (ambient, diffuse, specular lighting, and hard shadows) rendered via MiniLibX.
- **C++ Modules & Ford-Johnson Algorithm (C++ — cpps):** Completed 42 C++ Modules 00–09 (OOP, Orthodox Canonical Form, polymorphism, templates, STL); implemented the **Ford-Johnson Merge-Insert Sort algorithm (CPP09)** optimizing comparison counts using multiple STL sequence containers (std::vector, std::deque) and Jacobsthal sequence partitioning.
- **Foundations & Algorithms:** [push_swap](#) (two-stack sorting optimization), [pipex](#) (pipeline fork/dup2), [fract-ol](#) (Mandelbrot/Julia sets), [minitalk](#) (signal IPC), [libft](#), [ft_printf](#), [get_next_line](#). *In progress:* [webserv](#) (C++98 HTTP/1.1 epoll server), [transcendence](#).

PROGRAMMINGADVICES ROADMAP — 17/24 COURSES COMPLETE

Dr. Mohammed Abu-Hadhoud — algorithms & problem solving, C++, OOP, data structures, C#, SQL Server & T-SQL; finishing ADO.NET connectivity

- **Bank & ATM Management Engine (C++, 1,593 lines — Bank-and-ATM-System-CPP):** Engineered an inheritance-based OOP architecture featuring bitmask-based role permissions (bitwise flags), custom delimiter-based file serialization (###), and transaction audit logging.
- **Contacts Management System (C#, ADO.NET, SQL Server — Contacts-App-3Tier):** Built an enterprise 3-tier architecture (Presentation / Business / Data Access layers) with parameterized T-SQL stored procedures to enforce data integrity and prevent SQL injection.
- **Windows Forms Desktop Suite (C#, .NET):** Built event-driven desktop applications with real-time UI state evaluation and dynamic price recalculation on user input.
- **String & Date Utility Libraries (C++):** Built reusable libraries with overloaded static/instance methods, using MSVC property extensions to expose modern C#-style getter/setter syntax.

Also: 5 relational database schema designs (e-commerce, clinic management) • 215+ algorithmic solutions & data structure implementations